

Decidable and enumerable languages

Exercise sheet 2 (solutions)

Exercise 2.1. Is every decidable language enumerable? And vice versa?

Solution. Every decidable language is enumerable. Let `decide_L` decide $L \subseteq \Sigma^*$. Then the following program enumerates L :

```
def enum_L():
    for w in enum_words():      # iterate through all the words
        if decide_L(w) == 1:
            print(w)
```

The converse is false: HALT is enumerable but undecidable. □

Exercise 2.2. Prove that every infinite enumerable language can be enumerated without repetitions.

Proof. Let `enum_L` enumerate an infinite $L \subseteq \Sigma^*$. The following program enumerates L without repetitions:

```
def enum_L_no_rep():
    printed = []
    for a in enum_L():
        if a not in printed:
            printed.append(a)
            print(a)
```

□

Exercise 2.3. Which of the following languages are decidable/enumerable?

- (a) The set of all prime numbers (encoded in binary).
- (b) The set of numbers $C = \{x \cdot y \mid x \in A, y \in B\} \subseteq \mathbb{N}$, where $A, B \subseteq \mathbb{N}$ are decidable (respectively, enumerable).
- (c) A language $A \subseteq B$, where $B \subseteq \Sigma^*$ is decidable (respectively, enumerable).

Solution. (a) Decidable (see Lecture 2), and therefore enumerable by Exercise 2.1.

- (b) We first deal with decidability. If A or B is empty, then C is empty, hence decidable. If both A and B are nonempty and decided by `decide_A` and `decide_B`, respectively, the following program decides C :

```
def decide_C(z):
    if z == 0:
        if decide_A(0) or decide_B(0): # A or B contains 0
            return 1
        else:
            return 0
    for (x, y) in factors(z): # all x * y == z
        if decide_A(x) and decide_B(y):
            return 1
    return 0
```

Remark. Note that the case split (one of A and B is empty vs. both are nonempty) is a *proof method*: we are not asking a program to test which case holds. Given any concrete decidable languages A and B , one of the two cases applies in reality, and the corresponding program decides C . What the proof establishes is that in every possible situation, such a program exists.

Now suppose A and B are enumerable. Run the two enumerators in parallel, and keep two lists of the numbers they have produced so far. Whenever A 's enumerator outputs a new a , print $a \cdot b$ for every b already in the list for B ; symmetrically, whenever B 's enumerator outputs a new b , print $a \cdot b$ for every a already in the list for A . Every product $x \cdot y$ with $x \in A$, $y \in B$ is eventually printed, and nothing else is printed, so C is enumerable.

- (c) Not necessarily. For example, Σ^* is decidable (by a program that always outputs 1) and enumerable. However, $\overline{\text{HALT}} \subseteq \Sigma^*$ is neither decidable, nor enumerable. $\square$

Exercise 2.4. Let A and B be two decidable languages. Which of the following languages are also decidable: $\overline{A} = \Sigma^* \setminus A$, $A \cap B$, $A \cup B$? The same question for enumerable A and B .

Proof. Let $A, B \subseteq \Sigma^*$ be decided by programs `decide_A` and `decide_B`. Then $\overline{A}$ is decided by flipping the answer of `decide_A`:

```
def decide_not_A(w):
    return 1 - decide_A(w)
```

Further, $A \cap B$ (respectively, $A \cup B$) is decided by running both deciders and combining the answers with 'and' (respectively, 'or').

Now let A and B be enumerable. Their union is enumerable: run the enumerators of A and B in parallel and print every word produced by either of them. The intersection is enumerable too:

run both enumerators in parallel, remember the words each has produced, and print a word w as soon as it has been produced by *both* enumerators. The complement, however, is not always enumerable: HALT is enumerable, while $\overline{\text{HALT}}$ is not. $\square$

Exercise 2.5. Which of the following languages are decidable/enumerable?

- (a) HALT-ON-EMPTY —programs that halt on empty input, ε .
- (b) HALT-ON-SOMETHING —programs that halt on some input.
- (c) NEVER-HALT —programs that never halt on any input.

Solution. All three properties are *semantic* (they depend only on the function computed by the program) and *non-trivial* (some programs have them, some do not). By Rice's theorem, none of the three languages is decidable.

However, we can also prove undecidability explicitly for each. Given $\langle \pi, w \rangle$, construct

```
def  $\rho_{\pi,w}(\mathbf{x})$ :      # the input is ignored
     $\pi(w)$               # run  $\pi$  on  $w$  (hardcoded into  $\rho$ )
    return 1
```

- (a) HALT-ON-EMPTY : the program ρ halts on ε if and only if π halts on w . Thus, if we could decide HALT-ON-EMPTY , we would be able to decide HALT :

```
def decide_halt( $\pi$ ,  $w$ ):
     $\rho_{\pi,w} = \text{construct}(\pi, w)$ 
    return decide_halt_on_empty( $\rho_{\pi,w}$ )
```

We say that we *reduced* HALT to HALT-ON-EMPTY . Such reductions are a common way to prove undecidability—note that Rice's theorem in Lecture 2 is also proven by a reduction from HALT .

- (b) HALT-ON-SOMETHING : again, $\rho_{\pi,w}$ halts on some input (namely, every input) if and only if π halts on w .
- (c) NEVER-HALT : and again, $\rho_{\pi,w}$ never halts iff π doesn't halt on w . Note the reversal: this is a reduction from $\overline{\text{HALT}}$ to NEVER-HALT .

Now on enumerability: HALT-ON-EMPTY is enumerable by dovetailing all programs on input ε ; HALT-ON-SOMETHING by dovetailing all programs on all inputs. In each case, a program is printed as one of the relevant processes terminates.

Since HALT-ON-SOMETHING is enumerable but not decidable, Post's theorem implies that NEVER-HALT is *not* enumerable. $\square$

Exercise 2.6. Suppose that for each $i \in \mathbb{N}$, the language $A_i \subseteq \Sigma^*$ is decidable. Is the union $\bigcup_{i \in \mathbb{N}} A_i$ necessarily decidable?

Solution. No. Fix any undecidable language $A \subseteq \Sigma^*$. Suppose $A = \{w_0, w_1, w_2, \dots\}$ (A is countable, as any language, so we can index its words with natural numbers). Let $A_i = \{w_i\}$, $i \in \mathbb{N}$. Then each A_i is decidable (as a finite set), but $\bigcup_{i \in \mathbb{N}} A_i = A$ is undecidable.

Similarly, if A is chosen to be non-enumerable, then each A_i is enumerable, yet their union is not. $\square$

Exercise 2.7. Prove that any two enumerable languages A, B can be decomposed into disjoint enumerable languages. That is, there exist enumerable languages $A', B' \subseteq \Sigma^*$ such that:

$$A' \subseteq A, \quad B' \subseteq B, \quad A' \cup B' = A \cup B, \quad \text{and} \quad A' \cap B' = \emptyset.$$

Proof. Let `enum_A` and `enum_B` be programs enumerating A and B . Run the two enumerators in parallel, keeping two lists `list_A'` and `list_B'`, initially empty. Then:

- each time `enum_A` outputs w : if w is already in `list_B'`, discard it; otherwise, add w to `list_A'` and print it.
- each time `enum_B` outputs w : if w is already in `list_A'`, discard it; otherwise, add w to `list_B'` and print it.

Let A' contain all the words eventually appearing in `list_A'` and B' —all those eventually appearing in `list_B'`. Intuitively A' contains words from $A \setminus B$ and those words from $A \cap B$ that `enum_A` outputs *faster* than `enum_B`, and analogously for B' . $\square$

A language $L \subseteq \Sigma^*$ is called **semidecidable** if there is a program π such that on input $w \in \Sigma^*$:

- if $w \in L$, then π halts and outputs 1;
- if $w \notin L$, then π does not halt.

Exercise 2.8. Prove that a language is enumerable if and only if it is semidecidable.

Proof. ($\Rightarrow$) Suppose L is enumerated by a program `enum_L`. If $L = \emptyset$, it is semidecided by `never_stop`. Otherwise, here is a semidecider:

```
def semidecide_L(w):
    for u in enum_L():          # enumerate all words of L
        if u == w:
            return 1            # w in L: halt and accept
```

If $w \in L$, then `enum_L` eventually prints w , so we halt; if $w \notin L$, then w is never printed, so we search forever.

($\Leftarrow$) Suppose L is semidecided by a program `semidecide_L`. Use dovetailing to run `semidecide_L` on each $w \in \Sigma^*$ in parallel. Print every word on which `semidecide_L` terminates—this will enumerate L . $\square$